

Section 13: Classes and OOP Basics **Concept Map**

The full path through this section: we open here before each concept, then dive in.



OOP: Keywords Covered

Section 13: Classes and OOP Basics. The keywords in bright green are the new ones we cover here.

Values

True False None

Logic & Membership

and or not in is

Conditionals

if elif else

Loops

for while break continue

Functions

def return lambda

Classes

class

Error Handling

try except finally raise
assert

Modules & Imports

import from as

Scope

global nonlocal

Context Managers

with

Cleanup & Flow

pass del

Generators & Async

yield async await

Soft Keywords

match case _ type

keyword New in this section **keyword** Covered earlier **keyword** Not covered yet

Classes: why objects at all?

Count the variables before you meet the syntax

Two products already need **six** loose variables that must be kept in step by hand. Ten products need thirty. A class is how you stop counting.

Loose variables

```
product1_name = "Laptop"
product1_price = 1000
product1_stock = 5
product2_name = "Mouse"
product2_price = 25
product2_stock = 200
```

Nothing ties these together. Delete one, misspell one, or pass the wrong pair to a function and **Python will not notice**.

6 variables for 2 products

30 variables for 10 products

1 class, for any number

One blueprint

```
class Product:
    store_name = "Pyzon Store"

p1 = Product()
p2 = Product()
print(p1.store_name)    # Pyzon Store
```

Describe a product **once**, then make as many as you like. Calling the class name builds an object.

Honest about what is still missing

No product has its OWN name or price yet -> **Step-02**

print(p1) shows <__main__.Product object at 0x...> -> **Step-05**

OOP: Why Objects? Your First Class

A **class** is a blueprint that describes one kind of thing, and an **object** is one real item built from that blueprint, like a **Product** blueprint and the two products Laptop and Mouse made from it.

#	CONCEPT	SAMPLE CODE
1	The problem loose variables Two products already need six separate variables.	<pre>product1_name = "Laptop" product1_price = 1000</pre>
2	Write a blueprint class Describe "a product" once and reuse it for every product.	<pre>class Product: store_name = "Pyzon Store"</pre>
3	Create an object Product() Call the class name like a function to build one instance.	<pre>p1 = Product() print(p1.store_name)</pre>
4	What is still missing print(p1) Both share the store name, but neither has its own name yet.	<pre>print(p1) print(p2)</pre>

Classes: the `__init__` Method and the `self` Object

Not a keyword, not magic: just a name for the object being worked on

`__init__` runs by itself the moment you create an object, and `self` is that object.

Print it from inside, print it from outside

```
class Product:
    def __init__(self):
        print("inside __init__:", self)
inside __init__: <__main__.Product object at 0x1043b46e0>
```

```
p1 = Product()
print("outside __init__:", p1)
outside __init__: <__main__.Product object at 0x1043b46e0>
```

The same address, both times: one object, two names

the digits differ on every run; what matters is that they MATCH

```
def __init__(self, name, price, stock):
    self.name = name
    self.price = price
    self.stock = stock
```

now each object owns its data

```
p1 = Product("Laptop", 1000, 5)
p2 = Product("Mouse", 25, 200)
print(p1.name, p1.price, p1.stock)
print(p2.name, p2.price, p2.stock)
Laptop 1000 5
Mouse 25 200
```

`self.name = name` reads oddly until you see it: the right side is the parameter that was passed in, the left side is where it gets stored on this object.

OOP: Give Each Object Its Own Data

`__init__` is the setup method Python runs for you when an object is created, **self** is that object, and an **instance attribute** is data stored on it, so **Product("Laptop", 1000, 5)** makes a product with its own name, price, and stock.

#	CONCEPT	SAMPLE CODE
1	Setup method <code>__init__</code> Python runs it for you the moment an object is created.	<pre>class Product: def __init__(self):</pre>
2	The object itself <code>self</code> Inside and outside print the SAME object.	<pre>print("inside __init__:", self) print("outside __init__:", p1)</pre>
3	Start with own data <code>__init__(self, name)</code> Add a parameter, then pass a value when you create the object.	<pre>p1 = Product("Laptop") p2 = Product("Mouse")</pre>
4	Each object's own data <code>self.name = name</code> Data stored on self belongs to THIS object; each object keeps its own copy.	<pre>self.name = name self.price = price</pre>

Classes: **Class Attributes** vs **Instance Attributes**

Ask where the value should LIVE

Every product is sold by the same store, so the store name goes on the **class**, written once. A name, a price and a stock differ per product, so they go on **each object**.

CLASS ATTRIBUTE: one copy, shared by every product

```
class Product:  
    store_name = "Pyzon Store"
```

```
print(Product.store_name)    # Pyzon Store  
print(p1.store_name)        # Pyzon Store  
print(p2.store_name)        # Pyzon Store  
from the class or any object: the same shared value
```

INSTANCE ATTRIBUTES: each object keeps its own name, price and stock

```
p1 = Product("Laptop", 1000, 5)  
p1.price = 900  
print(p1.name, p1.price)  
Laptop 900  
only the Laptop's price changed
```

```
p2 = Product("Mouse", 25, 200)  
  
print(p2.name, p2.price)  
Mouse 25  
untouched: its price lives on p2
```

SAME NAME ON BOTH? The object's own value wins, for that object only

```
p2.store_name = "ABC Sellers"
```

```
p2.store_name  
ABC Sellers
```

```
p1.store_name  
Pyzon Store
```

```
Product.store_name  
Pyzon Store
```

OOP: Class vs Instance Attributes

A **class attribute** is written in the class body and is shared by every object, while an **instance attribute** is set on **self** and belongs to one object only: every Product shares the same **store_name**, but Laptop and Mouse each keep their own **price** and **stock**.

#	CONCEPT	SAMPLE CODE
1	Class attribute <code>store_name</code> Write it once in the class body; every object shares it.	<pre>class Product: store_name = "Pyzon Store"</pre>
2	Read it three ways <code>Product.store_name</code> The class or any object gives the same shared value.	<pre>print(Product.store_name) print(p1.store_name) print(p2.store_name)</pre>
3	Instance attributes stay separate <code>p1.price</code> Each object holds its own values; changing one leaves the rest alone.	<pre>p1.price = 900 print(p2.price) # still 25</pre>
4	Instance attribute wins <code>p2.store_name</code> Set the same name on an object and that object gets its own value.	<pre>p2.store_name = "ABC Sellers" print(p2.store_name) # instance attribute wins</pre>

Classes: Adding Behavior with **Methods**

A method is just a function that gets the object handed to it

Its first parameter is **self**, so it can read **this** object's data and change it. You call it on an object with a dot, like **p1.describe()**.

Read the object's data

```
def describe(self):  
    return f"{self.name} costs ${self.price}"
```

```
print(p1.describe())    # Laptop costs $1000  
print(p2.describe())    # Mouse costs $25
```

Change the object's data

```
def restock(self, qty):  
    self.stock += qty
```

```
def sell(self, qty):  
    self.stock -= qty
```

```
print(p1.stock)        # 5  
p1.restock(10)  
p1.sell(2)  
print(p1.stock)        # 13
```

The trap: without the parentheses, nothing runs and nothing complains

```
print(p1.describe)  
<bound method Product.describe of ...>  
you only named the method; you never ran it
```

```
print(p1.describe())  
Laptop costs $1000  
the () is what actually calls it
```

A method that **RETURNS** gives you something to keep working with, like **label.upper()** after **label = p1.describe()**. One that only **ACTS**, like **sell()**, hands back **None**, and that is correct.

OOP: Adding Behavior with Methods

A **method** is a function written inside a class, and its first parameter **self** is the object itself, so **describe()** and **sell()** can use and change that product's own data.

#	CONCEPT	SAMPLE CODE
1	Method with self <code>self</code> A function inside the class; self reaches that object's own data.	<pre>def describe(self): return f"{self.name} costs \${self.price}"</pre>
2	Change the data <code>self.stock -= qty</code> A method can also update the object's own data.	<pre>def sell(self, qty): self.stock -= qty</pre>
3	Return or act <code>return</code> <code>None</code> describe hands text back; sell only acts, so it returns None.	<pre>label = p1.describe() print(p1.sell(2)) # returns None</pre>
4	The () runs it <code>()</code> Without () you only name the method; with () you run it.	<pre>print(p1.describe) # not the text print(p1.describe()) # the text</pre>

Classes: Readable Objects with `__str__` and `__repr__`

`print()` and `repr()` each look for one exact method name

`print(p1)` uses `__str__` and `repr(p1)` uses `__repr__`. A method with any other name, like `describe()`, is never used by them.

A normal method: `print()` ignores it

```
def describe(self):  
    return f"{self.name} - ${self.price} ({self.stock} in stock)"
```

```
print(p1)  
<__main__.Product object at 0x...>  
print(p1.describe())  works only when WE call it  
Laptop - $1000 (5 in stock)
```

`__str__`: same body, exact name, the friendly form

```
def __str__(self):  
    return f"{self.name} - ${self.price} ({self.stock} in stock)"
```

```
print(p1)  
Laptop - $1000 (5 in stock)
```

`__repr__`: its own exact name, the exact form for developers

```
def __repr__(self):  
    return f"Product(name={self.name!r}, price={self.price!r}, stock={self.stock!r})"
```

```
print(repr(p1))  before adding __repr__  
<__main__.Product object at 0x...>  
print(repr(p1))  after adding __repr__  
Product(name='Laptop', price=1000, stock=5)
```

A list of products shows `__repr__`, one product shows `__str__`

```
print(products)  [Product(name='Laptop', price=1000, stock=5), Product(name='Mouse', price=25, stock=200)]  
print(products[0])  Laptop - $1000 (5 in stock)
```

A list shows each item with `__repr__`, so printing a list gives you the exact form. That is what you want when you are **debugging**.

OOP: Readable Objects with `__str__` and `__repr__`

Two more **dunder methods**, both about SHOWING an object. `print()` calls `__str__` for a friendly line; `repr()` and a list of objects use `__repr__` for the exact one. The name must match exactly; a normal method will not do.

#	CONCEPT	SAMPLE CODE
1	Plain object prints an address <code>print(p1)</code> With no <code>__str__</code> , Python shows the type and a memory address.	<pre>p1 = Product("Laptop", 1000, 5) print(p1)</pre>
2	A normal method is ignored <code>describe()</code> It works when YOU call it, but <code>print()</code> never looks for it.	<pre>def describe(self): print(p1.describe())</pre>
3	Friendly form <code>__str__</code> Same body, exact name, and now <code>print()</code> uses it.	<pre>def __str__(self): return f"{self.name} - \${self.price}"</pre>
4	<code>repr()</code> has a default too <code>repr(p1)</code> On a plain object, <code>str</code> and <code>repr</code> fall back to the SAME address.	<pre>print(p1) print(repr(p1))</pre>
5	Exact developer form <code>__repr__</code> A precise line that names each field, for inspecting objects.	<pre>return f"Product(name={self.name!r})"</pre>
6	A list shows <code>__repr__</code> <code>not __str__</code> Print a list to debug and you get the exact form, not the friendly one.	<pre>products = [p1, p2] print(products)</pre>

Classes: what a **decorator** is, and why we need one

A decorator is a line that starts with @, written just above a def

It does not run your code. It tells Python how that method should be used. **@classmethod** and **@staticmethod** are two that Python already gives you.

The shape

```
@classmethod
def from_dict(cls, data):
    return cls(data["name"], ...)
```

The **@ line** decorates the method written under it. Nothing else changes.

What it decides: who arrives first

no decorator	def describe(self):	Python sends the object
@classmethod	def from_dict(cls, data):	Python sends the class
@staticmethod	def is_valid_price(price):	Python sends nothing

Why we need it: a helper on the class must work from the class AND from an object

```
def is_valid_price(price):    # no decorator
Product.is_valid_price(1000)  True
p1.is_valid_price(1000)      TypeError: takes 1 positional
argument but 2 were given

Called on an object, Python puts p1 in the first slot, so price gets 1000 as a second value.
```

```
@staticmethod
def is_valid_price(price):
Product.is_valid_price(1000)  True
p1.is_valid_price(1000)      True

The decorator tells Python to send nothing first, so both calls mean the same thing.
```

A decorator is a **label Python reads before the method**. In this step it decides one thing: what arrives first, the object, the class, or nothing.

Classes: Class Methods and Static Methods

Look at the decorator and the first parameter: they tell you what the method receives

A normal method receives **self**, one object. A **@classmethod** receives **cls**, the class itself. A **@staticmethod** receives **neither**.

cls @classmethod receives the class and builds the object

```
@classmethod
def from_dict(cls, data):
    print("cls is:", cls)
    return cls(data["name"], data["price"], data["stock"])
```

```
p1 = Product.from_dict(data)
cls is: <class '__main__.Product'>
print(p1.name, p1.price, p1.stock)
Laptop 1000 5
```

self Both ways give a real Product that normal methods work on

```
def describe(self):
    return f"{self.name} - ${self.price}"

p1 = Product("Mouse", 25, 200)
p2 = Product.from_dict({"name": "Laptop", "price": 1000, "stock": 5})
```

```
print(p1.describe())    made directly
Mouse - $25
print(p2.describe())    made by from_dict
Laptop - $1000
```

neither @staticmethod is a related helper: no self, no cls

```
@staticmethod
def is_valid_price(price):
    return price > 0
```

```
print(Product.is_valid_price(1000))
True
print(Product.is_valid_price(-5))
False
```

Which one do I write? Need **the class itself**, like building a new object? Use **@classmethod**, because it gets **cls**. Just a related helper that needs no object and no class? Use **@staticmethod**.

OOP: Class Methods and Static Methods

Most methods work on one object through **self**. A **@classmethod** receives the class as **cls** (great for an alternate constructor like **from_dict**); a **@staticmethod** takes neither **self** nor **cls**, making it a related helper like **is_valid_price**.

#	CONCEPT	SAMPLE CODE
1	classmethod builds an object cls It gets the CLASS as cls, so cls(...) makes a new Product.	<pre>@classmethod def from_dict(cls, data): p1 = Product.from_dict(data1)</pre>
2	staticmethod, no self/cls @staticmethod A related helper that lives on the class but needs no object.	<pre>@staticmethod def is_valid_price(price):</pre>
3	Which one do I write? cls neither Need the class itself -> classmethod; just a helper -> staticmethod.	<pre>Product.from_dict(data) Product.is_valid_price(data["price"])</pre>